

Week 16 Assignment

Real-Time Chat Application with Socket.io

Ronish Shrestha

Objective

In this exercise, we will build a real-time chat application with additional features.

We will:

- establish WebSocket connections
- send and receive messages
- broadcast messages
- track user join/leave events
- include timestamps in messages

Step 1 — Create Project Folder

Create a new folder for the project:

```
mkdir week16-chat-app  
cd week16-chat-app
```

Step 2 — Initialize Project

Run the following command to initialize the project:

```
npm init -y
```

Step 3 — Install Dependencies

Install Express and Socket.io:

```
npm install express socket.io
```

Step 4 — Create Server

Create a file called server.js and add the following base setup code:

```
const express = require('express');  
const http = require('http');  
const { Server } = require('socket.io');
```

```

const path    = require('path');

const app     = express();
const server  = http.createServer(app);
const io      = new Server(server);

// Serve the frontend HTML file
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});

const PORT = 5000;
server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Step 5 — Handle Socket Connections

Add this code to server.js to handle socket connections:

```

io.on('connection', (socket) => {
  console.log('A user connected:', socket.id);

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});

```

Step 6 — Create Frontend (index.html)

Create index.html. HTML makes the website layout and JavaScript provides the interaction and functionality.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Real-Time Chat</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 600px;
           margin: 40px auto; padding: 0 20px; }
    ul { list-style: none; padding: 0; }
    li { padding: 6px 0; border-bottom: 1px solid #eee; }
    .system { color: #888; font-style: italic; }
    input { padding: 8px; margin-right: 6px; width: 200px; }
    button { padding: 8px 14px; cursor: pointer; }
  </style>
</head>
<body>
  <h2>Real-Time Chat</h2>

  <div id="join-section">
    <input id="username" placeholder="Enter username" />
    <button onclick="joinChat()">Join Chat</button>
  </div>

  <div id="chat-section" style="display:none;">

```

```

<ul id="messages"></ul>
<input id="message" placeholder="Type a message..." />
<button onclick="sendMessage()">Send</button>
</div>

<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();

  function joinChat() {
    const username = document.getElementById('username').value.trim();
    if (!username) return;
    socket.emit('join', username);
    document.getElementById('join-section').style.display = 'none';
    document.getElementById('chat-section').style.display = 'block';
  }

  function sendMessage() {
    const username = document.getElementById('username').value.trim();
    const message = document.getElementById('message').value.trim();
    if (!message) return;
    const timestamp = new Date().toLocaleTimeString();
    socket.emit('chat', { username, message, timestamp });
    document.getElementById('message').value = '';
  }

  socket.on('chat', (data) => {
    const li = document.createElement('li');
    li.textContent = `[${data.timestamp}] ${data.username}: ${data.message}`;
    document.getElementById('messages').appendChild(li);
  });

  socket.on('system', (msg) => {
    const li = document.createElement('li');
    li.textContent = msg;
    li.className = 'system';
    document.getElementById('messages').appendChild(li);
  });
</script>
</body>
</html>

```

Explanation

- **Library Loading:** The `<script>` tag fetches the Socket.IO client library directly from your Node server.
- **Connection Initialization:** `const socket = io()` establishes a real-time two-way connection between the browser and server.
- **Sending Data:** `sendMessage()` captures user input and uses `socket.emit('chat', {...})` to send username, message, and timestamp.
- **Receiving Chat:** `socket.on('chat', ...)` listens for messages from the server and appends them to the messages list.
- **System Alerts:** `socket.on('system', ...)` handles join/leave notifications, styled in grey to separate them from user messages.

Step 7 — Broadcast Messages

Update server.js to listen for chat messages and broadcast them to all connected clients:

```
socket.on('chat', (data) => {  
  // Broadcast the message to ALL connected clients  
  io.emit('chat', data);  
});
```

Step 8 — Join & Leave Notifications

Update server.js to add join and leave notifications. Below is the complete updated server.js:

```
const express = require('express');  
const http    = require('http');  
const { Server } = require('socket.io');  
const path     = require('path');  
  
const app      = express();  
const server   = http.createServer(app);  
const io       = new Server(server);  
  
app.get('/', (req, res) => {  
  res.sendFile(path.join(__dirname, 'index.html'));  
});  
  
io.on('connection', (socket) => {  
  console.log('A user connected:', socket.id);  
  
  socket.on('join', (username) => {  
    socket.username = username;  
    socket.broadcast.emit('system', `${username} joined the chat`);  
  });  
  
  socket.on('chat', (data) => {  
    io.emit('chat', data);  
  });  
  
  socket.on('disconnect', () => {  
    if (socket.username) {  
      io.emit('system', `${socket.username} left the chat`);  
    }  
  });  
});  
  
const PORT = 5000;  
server.listen(PORT, () => {  
  console.log(`Server running on http://localhost:${PORT}`);  
});
```

Step 9 — Update Frontend to Send Join Event

The joinChat() function in index.html already emits the join event (added in Step 6):

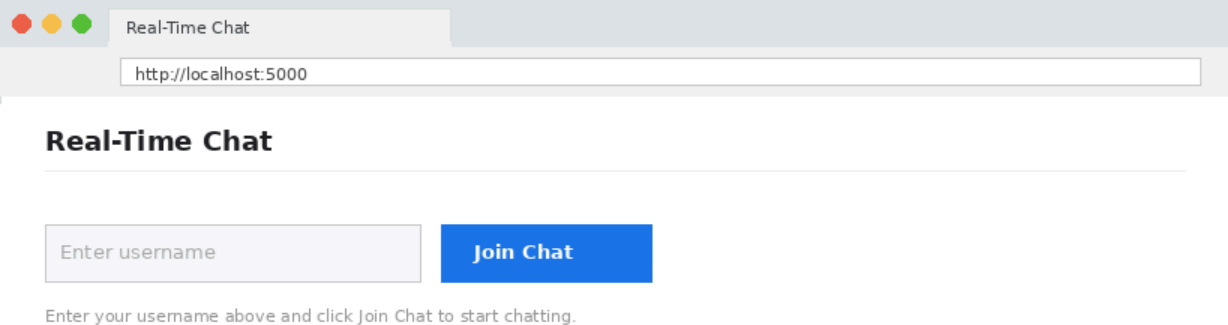
```
function joinChat() {  
  const username = document.getElementById('username').value.trim();  
  if (!username) return;  
  socket.emit('join', username);  
  document.getElementById('join-section').style.display = 'none';  
  document.getElementById('chat-section').style.display = 'block';  
}
```

Step 10 — Test the Application

Start the server and open the app in your browser:

```
node server.js  
  
# Server running on http://localhost:5000
```

Open `http://localhost:5000` in your browser. You will see the join screen:



The screenshot shows a web browser window with the title "Real-Time Chat". The address bar contains "http://localhost:5000". The page content features the heading "Real-Time Chat" followed by a horizontal line. Below this is a form with a text input field labeled "Enter username" and a blue button labeled "Join Chat". At the bottom of the form, there is a small instruction: "Enter your username above and click Join Chat to start chatting."

Join screen — enter a username and click Join Chat

Step 11 — Test with Multiple Tabs

Open three browser tabs at `http://localhost:5000`. Use a different username in each tab to simulate a group chat.

- Tab 1: logged in as Alice
- Tab 2: logged in as Bob
- Tab 3: logged in as Carol

All tabs receive messages in real time. System notifications appear when a user joins.

Tab 1 — Alice's view

Real-Time Chat — Tab 1

http://localhost:5000

Real-Time Chat

✦ Bob joined the chat

✦ Carol joined the chat

[10:01:32 AM] Alice: Hello everyone! ☐

[10:01:45 AM] Bob: Hey Alice! How are you?

[10:02:03 AM] Carol: Hi both! This is so cool ☐

[10:02:20 AM] Alice: Real-time chat is working great!

[10:02:35 AM] Bob: Socket.io is amazing ☐

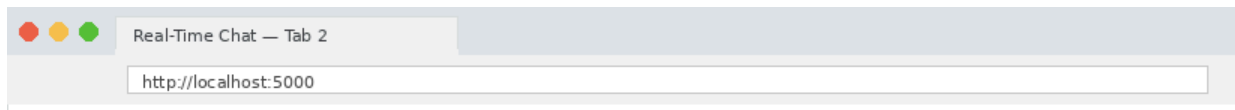
Type a message...

Send

Tab 1 — logged in as: Alice

Tab 1 (Alice) — sees Bob and Carol join, then live messages from all users

Tab 2 — Bob's view



Real-Time Chat

✦ Carol joined the chat

[10:01:32 AM] Alice: Hello everyone! ☺

[10:01:45 AM] Bob: Hey Alice! How are you?

[10:02:03 AM] Carol: Hi both! This is so cool ☺

[10:02:20 AM] Alice: Real-time chat is working great!

[10:02:35 AM] Bob: Socket.io is amazing ☺

[10:02:50 AM] Carol: All 3 tabs synced in real time!

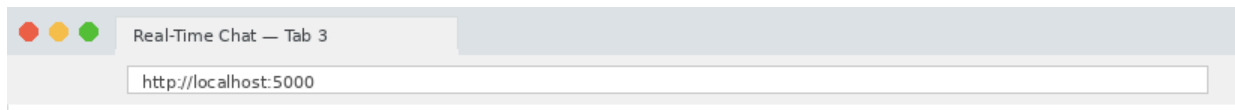
Type a message...

Send

Tab 2 — logged in as: Bob

Tab 2 (Bob) — receives all messages including Carol joining notification

Tab 3 — Carol's view



Real-Time Chat

[10:01:32 AM] Alice: Hello everyone! ☺
[10:01:45 AM] Bob: Hey Alice! How are you?
[10:02:03 AM] Carol: Hi both! This is so cool ☺
[10:02:20 AM] Alice: Real-time chat is working great!
[10:02:35 AM] Bob: Socket.io is amazing ☺
[10:02:50 AM] Carol: All 3 tabs synced in real time!

✦ Bob left the chat

Type a message...

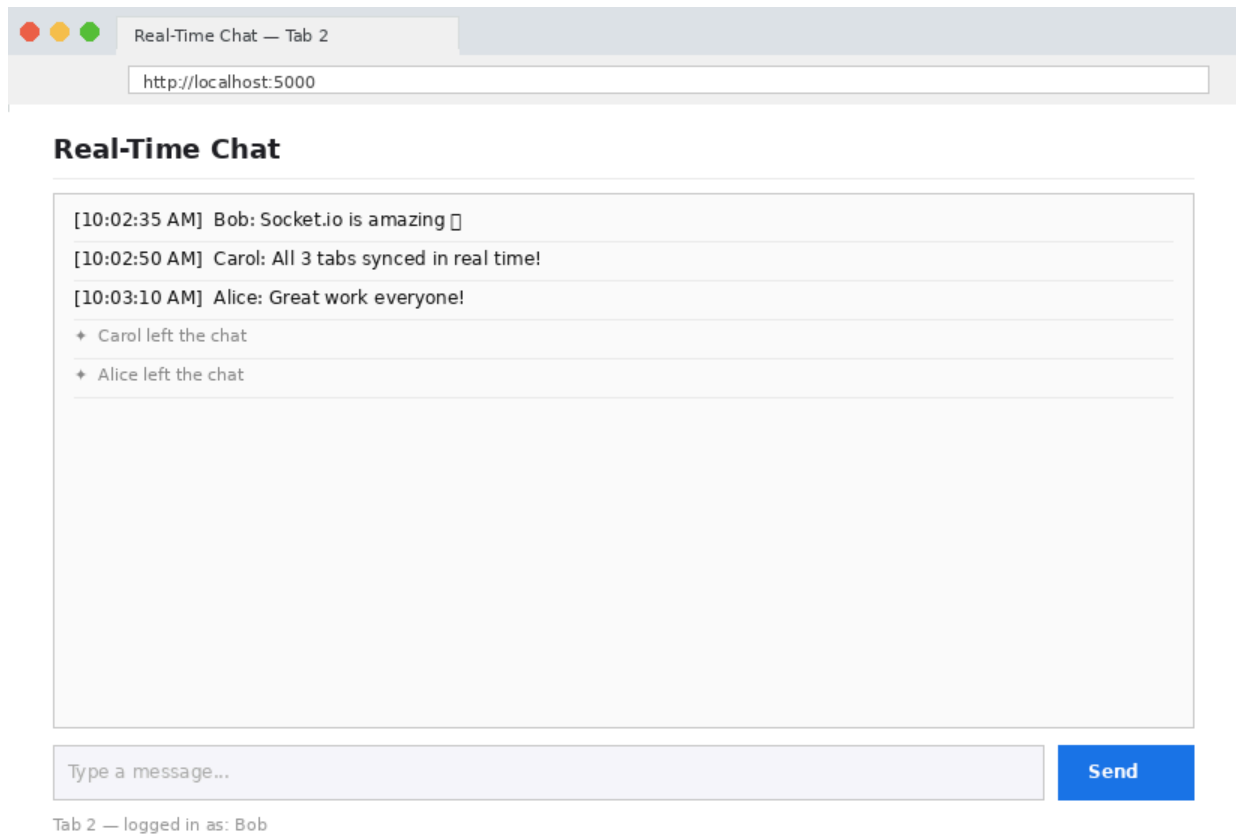
Send

Tab 3 — logged in as: Carol

Tab 3 (Carol) — sees all messages and Bob's leave notification after refresh

Step 12 — Test Leave Notification

Refresh any tab to disconnect that user. The other open tabs immediately show a leave notification in grey.



Leave notification — other tabs show 'user left the chat' in grey when a tab is refreshed

Expected Output

After testing, you should see:

- Real-time chat messages appearing across all tabs instantly
- Usernames displayed alongside each message
- Timestamps shown for every message
- "user joined" notification when a new user connects
- "user left" notification when a user refreshes or closes their tab

Learning Outcomes

We learned:

- WebSocket communication using Socket.io
- Event-based programming with `socket.emit()` and `socket.on()`
- Real-time data updates without page refresh
- Broadcasting messages to all connected clients using `io.emit()`
- Handling user connections and disconnections